Relatório 21 - Prática: Visão Computacional (III)

Matheus Beuren Carvalho

1. Introdução

O conteúdo do card se baseou nas redes convolucionais(CNN) aplicadas à biblioteca PyTorch, que ao contrário do TensorFlow as funções e definições de rede tem que ser feitas na mão. A seguir será mostrado como foi o desenvolvimento do card e quais os principais conteúdos abordados.

2. Desenvolvimento

O desenvolvimento do card iniciou-se na introdução básica a deep learning, pytorch e tensorflow. Diferente de outros cards, somente o pytorch é algo realmente novo, se baseia em um framework de aprendizado de máquina e deep learning feito para a linguagem python. A vantagem de usar o pytorch é que diferente do numpy que faz operações, o tensor permite que seja usado mais eficientemente as CPUs e processar em paralelo às operações.

A rede neural é formada pelos neurônios de entrada, camada de neurônios densa e a camada de saída, sendo conectadas por diversas linhas onde cada uma delas possui pesos para determinar o quanto vale aquela decisão. Essa divisão em camadas nos permite classificar objetos em uma ou múltiplas classes com base na função de ativação na camada final que retorna a probabilidade de cada classe. Um conceito muito importante é o backpropagation, em que o modelo prevê as probabilidades, calcula os erros e volta na rede ajustando os pesos. Nesse contexto, existem as funções de ativação que eliminam dados negativos assim transformando em uma rede não linear, e também a função de que compara a previsão com o valor real para depois um otimizador aplicar esses pesos às ligações entre cada camada.

Um conceito interessante de se alinhar é o DataLoader que de forma resumida organiza os dados do tensor em tuplas para que a rede consiga entender, diferente do TensorFlow que nao precisa de organizações claras igual ao Pytorch.

A introdução do vídeo a CNN se baseia praticamente no mesmo conceito dos cards passados, abordando as features maps, as camadas de pooling e o flatten, o diferencial é que agora será feito utilizando o pytorch e o openCV.

O Hyper parameter tuning é o processo de ajustar os parâmetros e uma rede neural, para assim encontrar os melhores parâmetros de uma rede. Com eles definidos os parâmetros podem variar desde a taxa de aprendizado até a

quantidade de neurônios em cada camada. Ele é importante para otimizar uma rede que tenha algum overfitting ou nao tem um bom desempenho.

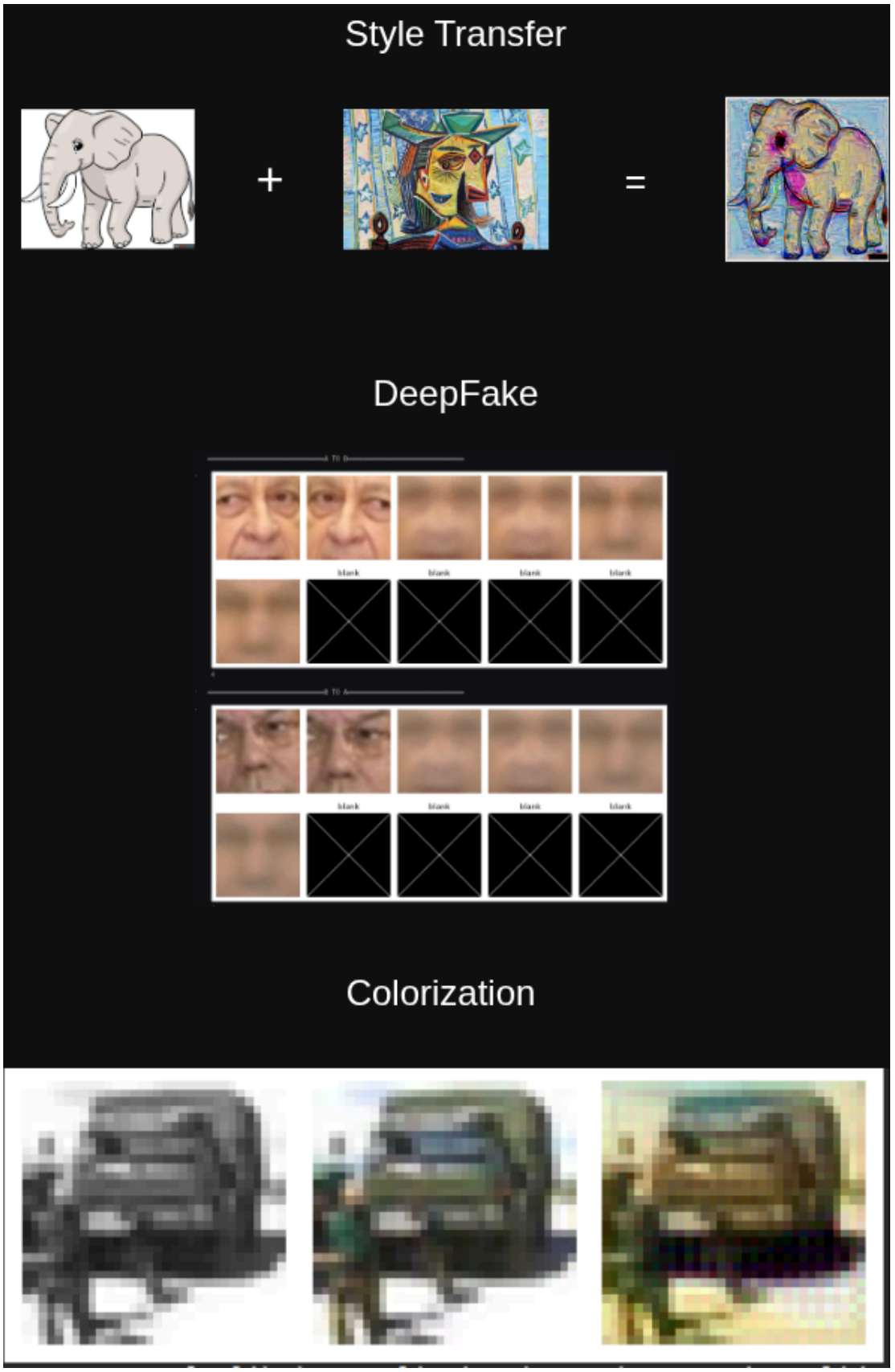
Data Argumentation se define pela criação de novas imagens de treino a partir das já existentes, e isso é feito através de modificações de rotação de imagem, pequenas alterações na imagem, ou seja modificações não no conteúdo da imagem mas sim como ela é vista. Porém um questionamento comum é, mas isso não afeta o treino e causa overfitting, na prática não pois a rede neural não sabe que é a mesma imagem, para nós humanos é fácil reconhecer, porém para a máquina ela não sabe que foi somente rotacionada.

A definição de AutoEncoders se baseia na ideia de uma rede neural aprender a codificar e decodificar imagens, onde é extraído somente o principal do objeto e diminui o ruído, ou seja, ela aprende com o que realmente importa para identificar o objeto. Suas características são o treino sem a necessidade de rótulos(aprendizado não supervisionado) e reduz o tamanho de imagens. Nesse contexto de autoencoders, o VAE se define por um criador de imagens com base no que aprendeu no treino, sendo mais técnico, ele aprende a organizar os dados em um espaço latente probabilísticos, assim sendo possível a reconstruir dados e gerar novos dados a partir de uma amostra aleatória.

Ao final do curso de Pytorch, foram apresentados três projetos diferentes mas muito aplicados na vida real. O Neural Style Transfer pega duas imagens, de uma pega o formato e conteúdo, e da segunda pega o estilo, assim gerando uma terceira imagem aplicando o conteúdo da primeira imagem com o estilo da segunda. Deepfake se resume a pegar duas imagens de rosto e troca-las, por exemplo, pessoa A trocar de rosto com a pessoa B. Por fim, o Image Colorization pega uma imagem preto e branco e prevê as cores dela.

Na figura 1 pode-se observar três projetos abordados no curso.

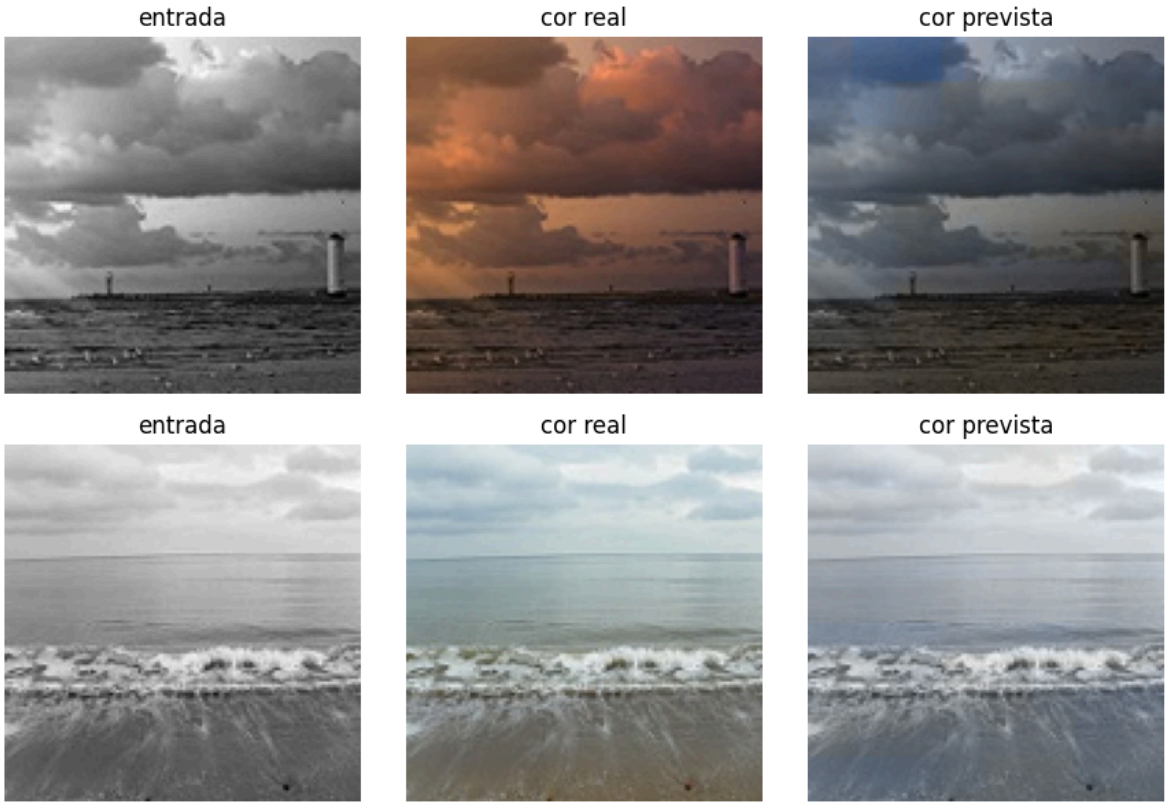
Figura 1 - Projetos



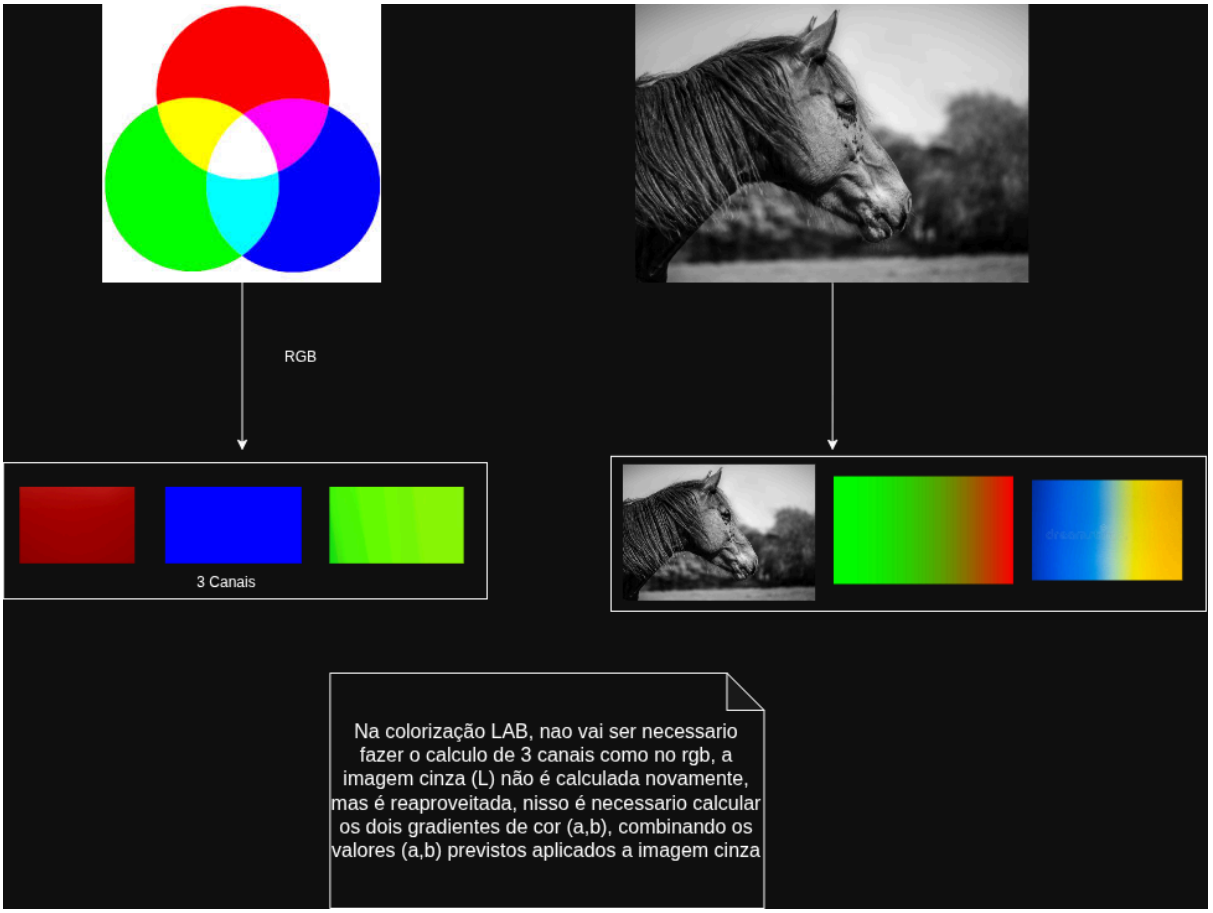
Na primeira parte do figura, podemos observar o projeto style transfer que copia o conteúdo da primeira imagem e pega o estilo da segunda, assim na terceira a imagem final. Depois temos a fumaça de rosto entre pessoas, a primeira linha é a imagem real, e a segunda a rede tentando trocar os rostos de duas pessoas. Finalizando, temos o colorization, a primeira imagem se refere a ela cinza, a segunda a imagem real, e a terceira a imagem que a rede prevê(cores).

Na Figura 2, o exemplo da prática é colorization, porém com um dataset muito maior que no de aula. Nisso, foi detectado um leve overfitting no modelo, mas ele conseguiu consideravelmente prever as cores das imagens.

Figura 2 - Prática colorization



Em vista da prática dos projetos propostos na aula, utilizei o Lab, que é um tipo de colorização de imagem que substitui nesse caso o RGB. O motivo de de realizar essa troca vem pela desnecessidade de recalcular todos os canais de entrada, onde com o RGB seria necessário a rede calcular para o output as cores verde, vermelho e azul, com o Lab não precisa, pois seu primeiro canal de entrada é a própria imagem em escala de cinza e como input do encoder ele vem com o brilho da imagem cinza. Na figura 3 é visto como é a diferença de cálculo entre os dois.



3. Conclusão

Concluindo o desenvolvimento do card, o conteúdo abordado refere-se a aplicação de CNNs para a biblioteca pytorch. Esta biblioteca demanda o manejo manual de classes e funções, diferente do TensorFlow onde o trabalho já é feito pela própria biblioteca. Porém, o PyTorch demanda mais trabalho, mas é muito mais flexível comparado ao TensorFlow.